

Data Structures and Algorithms

Lab Sheet 10 - Sorting and Searching Algorithms

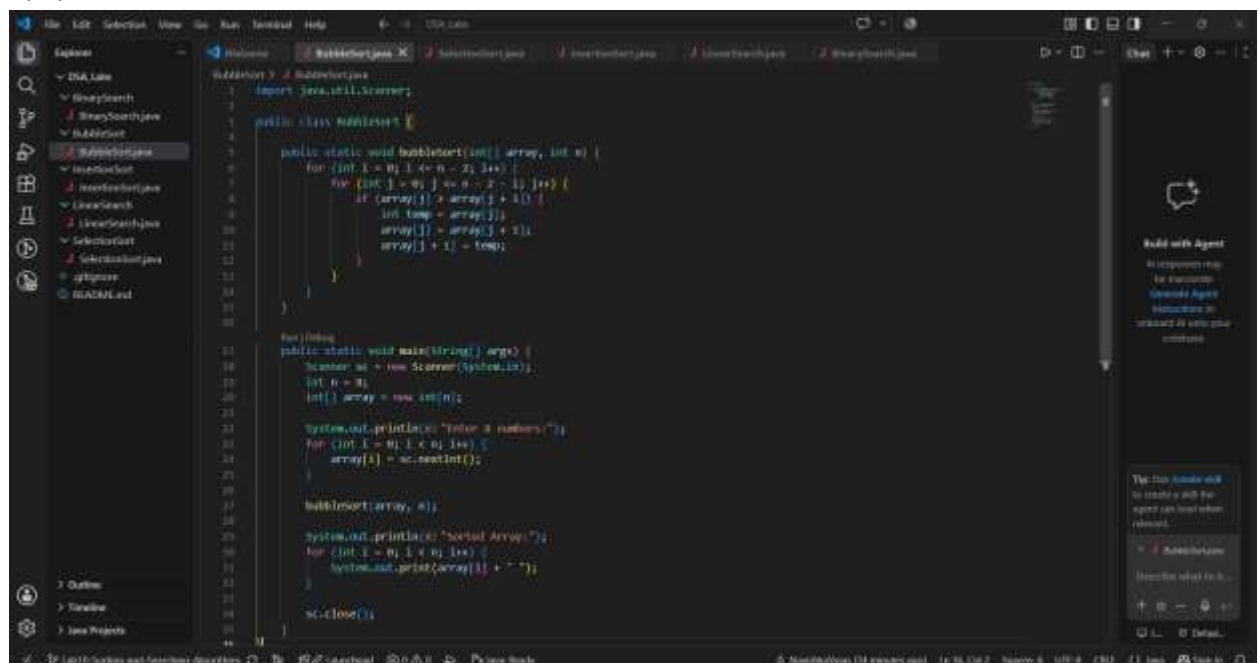
Cit-25-01-0369 - W.A.D.N.Y.Gunasekara

Part 01.

a) Algorithm BubbleSort(Array, n)

1. Start
2. Repeat for $i = 0$ to $n-2$
 - a. Repeat for $j = 0$ to $n-2-i$
 - If $\text{Array}[j] > \text{Array}[j+1]$
 - Swap $\text{Array}[j]$ and $\text{Array}[j+1]$
3. Display sorted Array
4. Stop

b).c).



```
import java.util.Scanner;

public class BubbleSort {

    public static void bubbleSort(int[] array, int n) {
        for (int i = 0; i < n - 1; i++) {
            for (int j = 0; j < n - i - 1; j++) {
                if (array[j] > array[j + 1]) {
                    int temp = array[j];
                    array[j] = array[j + 1];
                    array[j + 1] = temp;
                }
            }
        }
    }

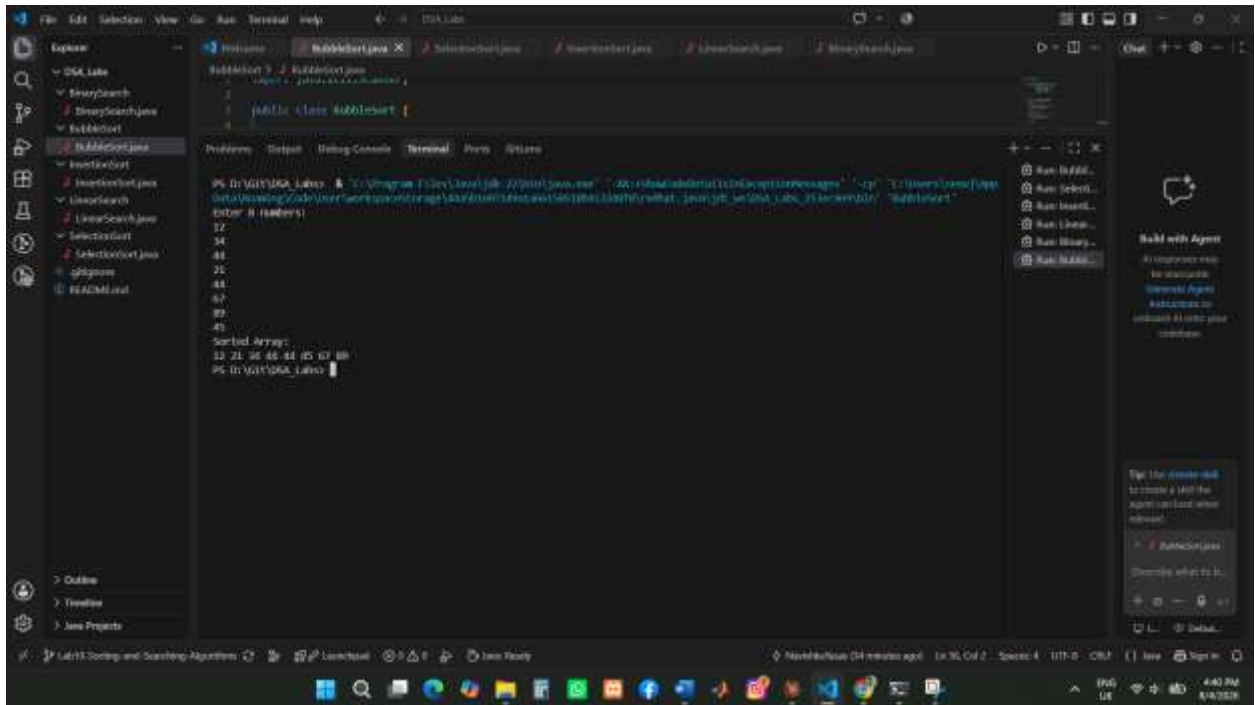
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = 0;
        int[] array = new int[n];

        System.out.println("Enter n numbers:");
        for (int i = 0; i < n; i++) {
            array[i] = sc.nextInt();
        }

        bubbleSort(array, n);

        System.out.println("Sorted Array:");
        for (int i = 0; i < n; i++) {
            System.out.print(array[i] + " ");
        }

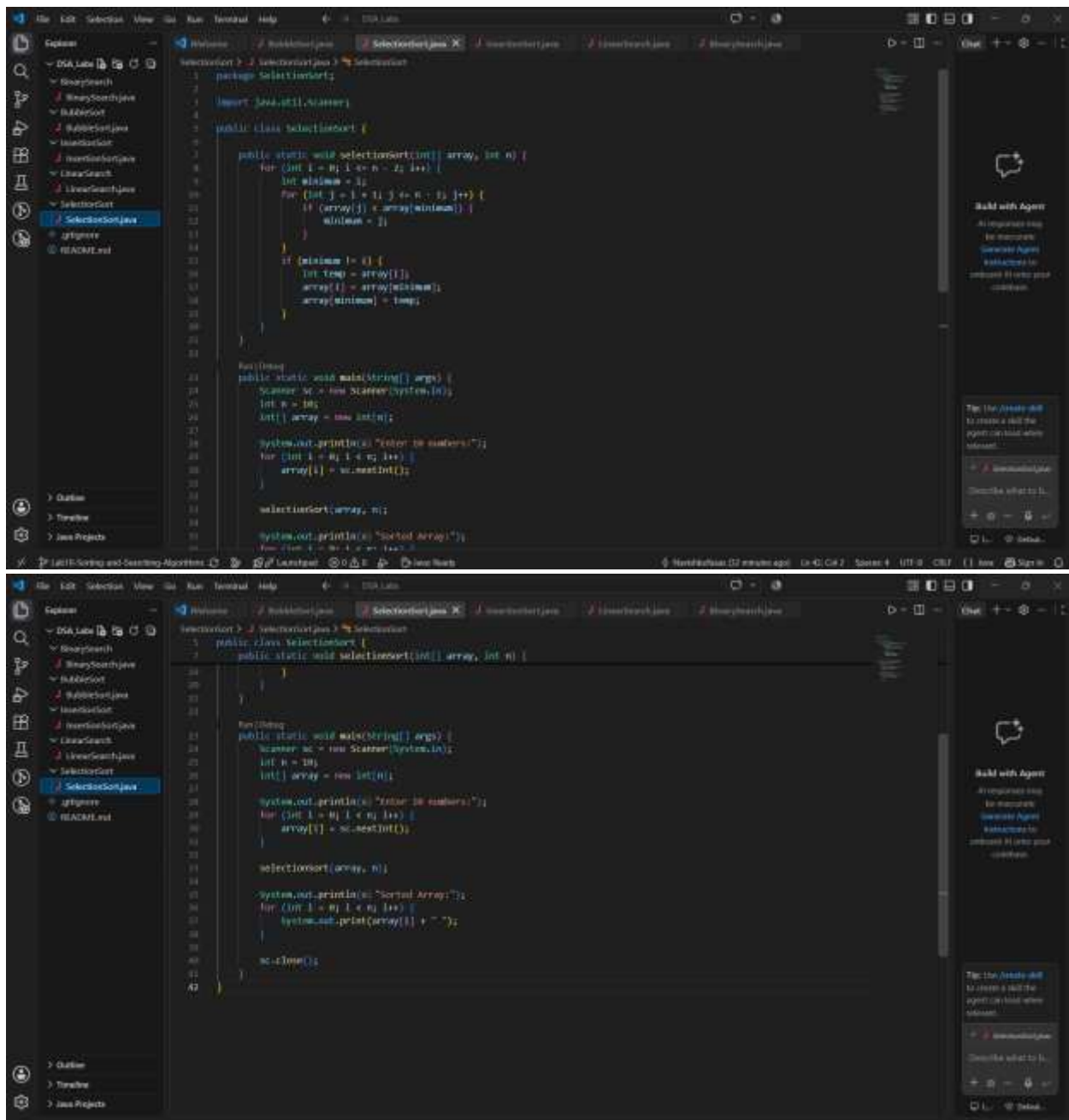
        sc.close();
    }
}
```

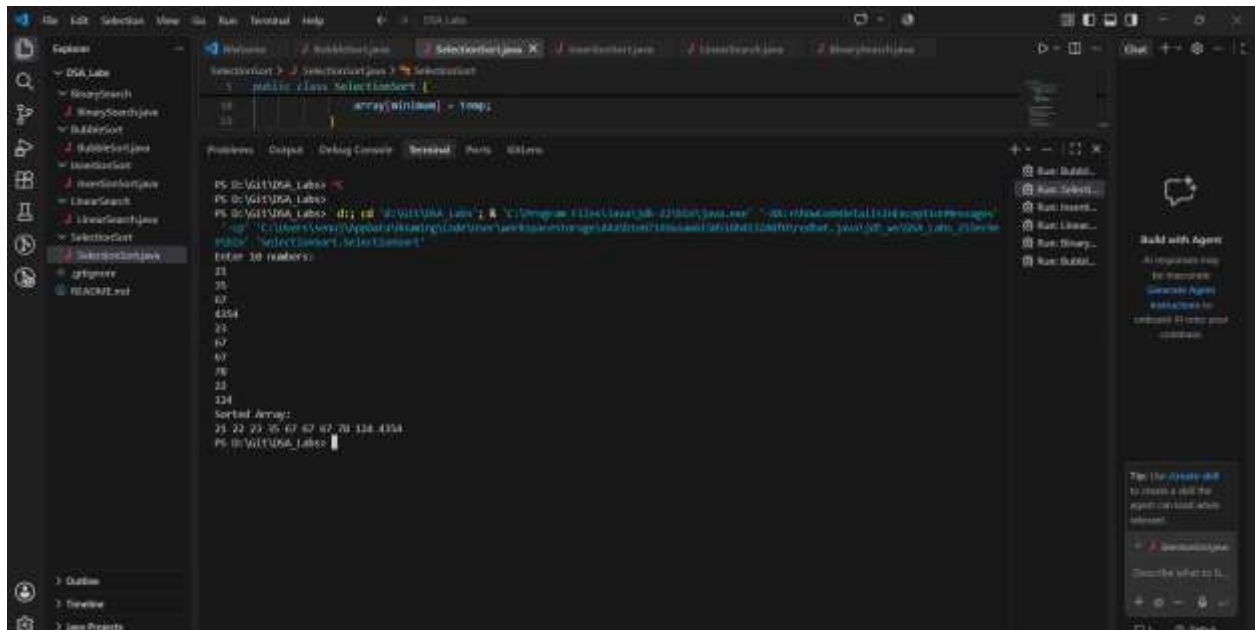


Part 02.

- a) Algorithm SelectionSort(Array, n)
 1. Start
 2. Repeat for $i = 0$ to $n-2$
 - a. Set minimum = i
 - b. Repeat for $j = i+1$ to $n-1$
If $\text{Array}[j] < \text{Array}[\text{minimum}]$
Set minimum = j
 - c. If minimum $\neq i$
Swap $\text{Array}[i]$ and $\text{Array}[\text{minimum}]$
 3. Display sorted Array
 4. Stop

b)c).





Part 03.

- a) Algorithm InsertionSort(Array, n)
 1. Start
 2. Repeat for $i = 1$ to $n-1$
 - a. Set $key = Array[i]$
 - b. Set $j = i-1$
 - c. Repeat while $j \geq 0$ and $Array[j] > key$
Set $Array[j+1] = Array[j]$
Set $j = j-1$
 - d. Set $Array[j+1] = key$
 3. Display sorted Array
 4. Stop

b)c)

```

1 package InsertionSort;
2
3 import java.util.Scanner;
4
5 public class InsertionSort {
6
7     public static void insertionSort(int[] array, int n) {
8         for (int i = 1; i <= n - 1; i++) {
9             int key = array[i];
10            int j = i - 1;
11            while (j >= 0 && array[j] > key) {
12                array[j + 1] = array[j];
13                j = j - 1;
14            }
15            array[j + 1] = key;
16        }
17    }
18
19    public static void main(String[] args) {
20        Scanner sc = new Scanner(System.in);
21        int n = 10;
22        int[] array = new int[n];
23
24        System.out.println("Enter 10 numbers:");
25        for (int i = 0; i < n; i++) {
26            array[i] = sc.nextInt();
27        }
28
29        insertionSort(array, n);
30
31        System.out.println("Sorted Array:");
32        for (int i = 0; i < n; i++) {
33            System.out.print(array[i] + " ");
34        }
35    }
36 }

```

The screenshot shows an IDE with a Java file named `InsertionSort.java`. The code defines a `Sorter` interface and an `InsertionSort` class that implements it. The `InsertionSort` class has a `sort` method that sorts an array using insertion sort and a `print` method that prints the array. The `main` method creates an array `int[] arr = {7, 12, 12, 23, 23, 34, 34, 56, 56, 56, 56, 454, 13};` and calls `sorter.sort(arr);` followed by `sorter.print(arr);`. The output window shows the sorted array: `7 12 12 23 23 34 34 56 56 56 56 454 13`.

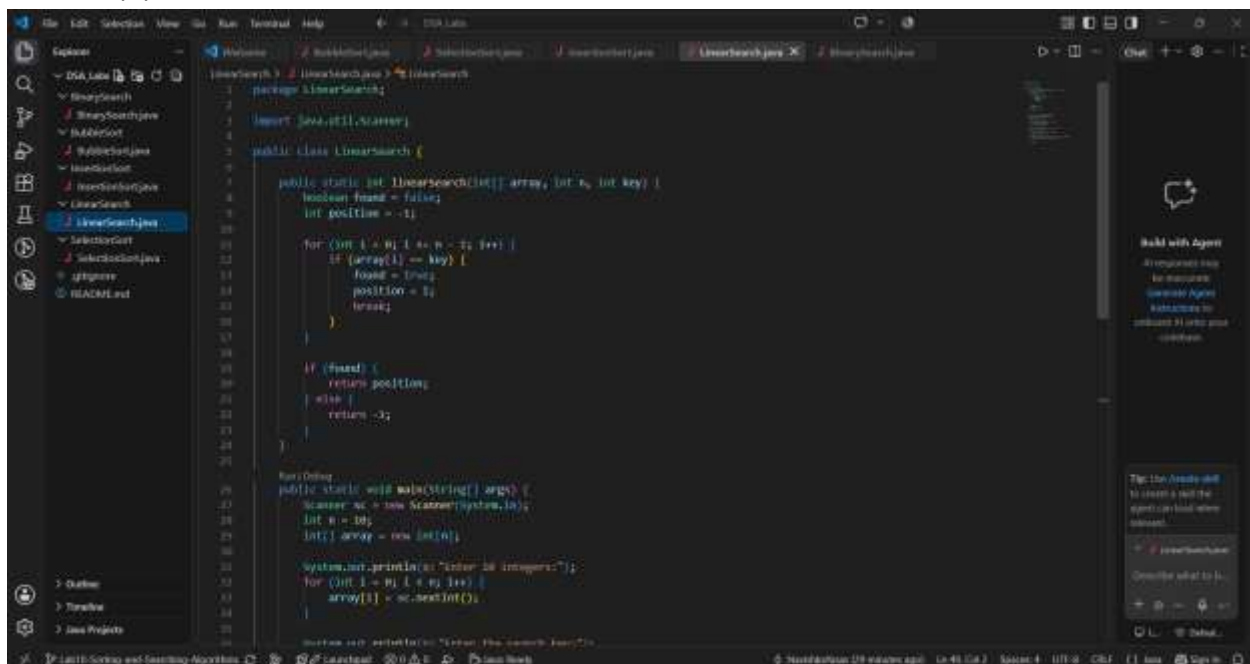
Part 04.

a) Algorithm LinearSearch(Array, n, key)

1. Start
2. Set found = false
3. Repeat for $i = 0$ to $n-1$

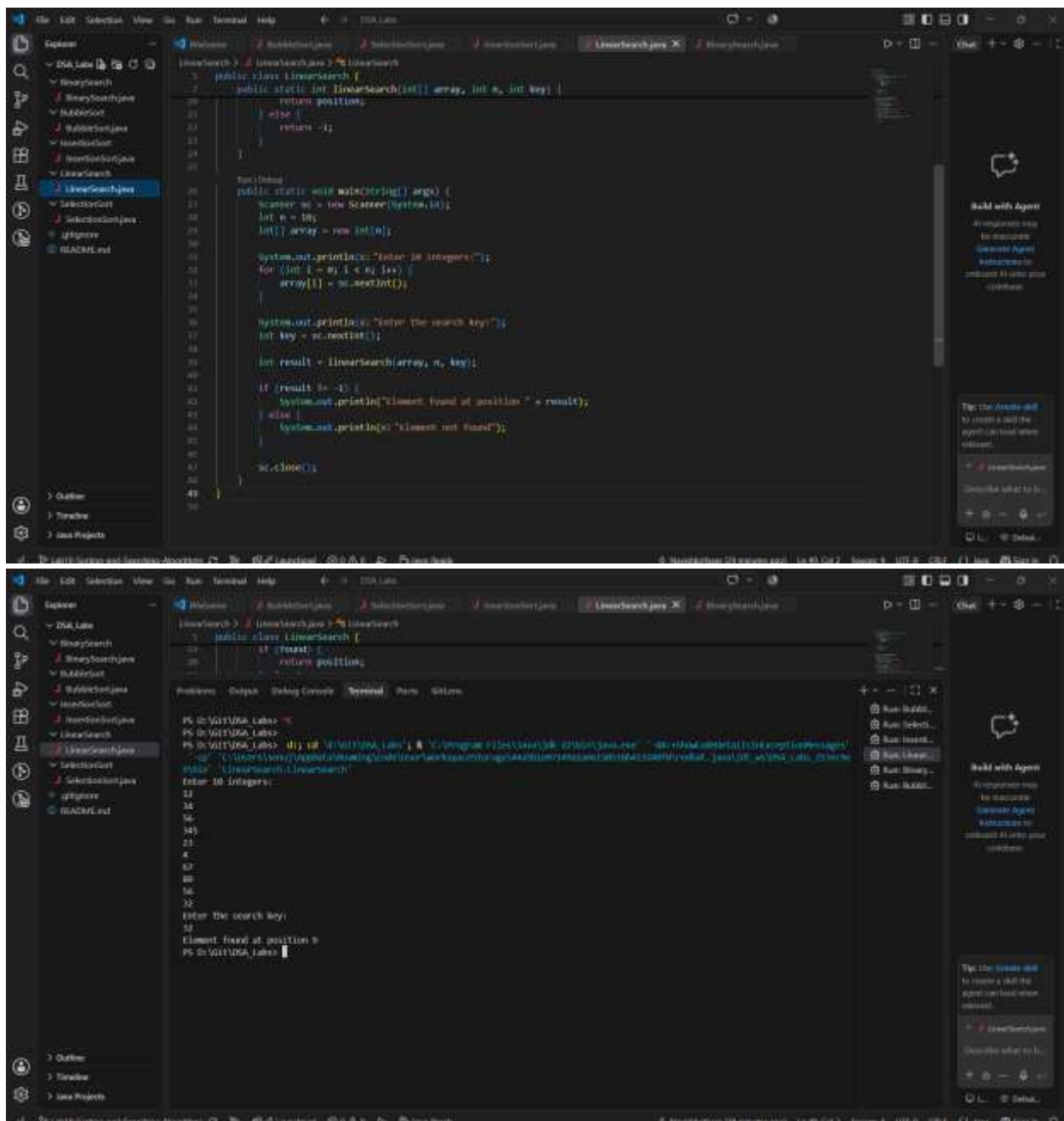
- a. If `Array[i] == key`
Set `found = true`
Set `position = i`
Exit loop
4. If `found == true`
Display "Element found at position", position
Else
Display "Element not found"
5. Stop

b)c)



The screenshot shows an IDE with a project named "DSA Labs". The file explorer on the left lists several Java files, with "LinearSearch.java" selected. The main editor displays the code for "LinearSearch.java". The code includes a package declaration, an import for the Scanner class, and a public class named "Linearsearch". The class contains a static method "linearsearch" that takes an array of integers, a key, and returns the position of the key if found, or -1 otherwise. A main method is also present, which prompts the user to enter the size of the array and the elements, then calls the "linearsearch" method and prints the result.

```
1 package LinearSearch;
2
3 import java.util.Scanner;
4
5 public class Linearsearch {
6
7     public static int linearsearch(int[] array, int k, int key) {
8         boolean found = false;
9         int position = -1;
10
11         for (int i = 0; i < array.length; i++) {
12             if (array[i] == key) {
13                 found = true;
14                 position = i;
15                 break;
16             }
17         }
18
19         if (found) {
20             return position;
21         } else {
22             return -1;
23         }
24     }
25
26     // Driver Code
27     public static void main(String[] args) {
28         Scanner sc = new Scanner(System.in);
29         int n = 10;
30         int[] array = new int[n];
31
32         System.out.println("Enter 10 integers:");
33         for (int i = 0; i < array.length; i++) {
34             array[i] = sc.nextInt();
35         }
36     }
37 }
```



Part 05.

a) Algorithm BinarySearch(Array, n, key)

1. Start
2. Set low = 0
3. Set high = n-1
4. Set found = false

5. Repeat while $low \leq high$ and $found == false$

a. Set $mid = (low + high) / 2$

b. If $Array[mid] == key$

Set $found = true$

Set $position = mid$

c. Else if $Array[mid] < key$

Set $low = mid + 1$

d. Else

Set $high = mid - 1$

6. If $found == true$

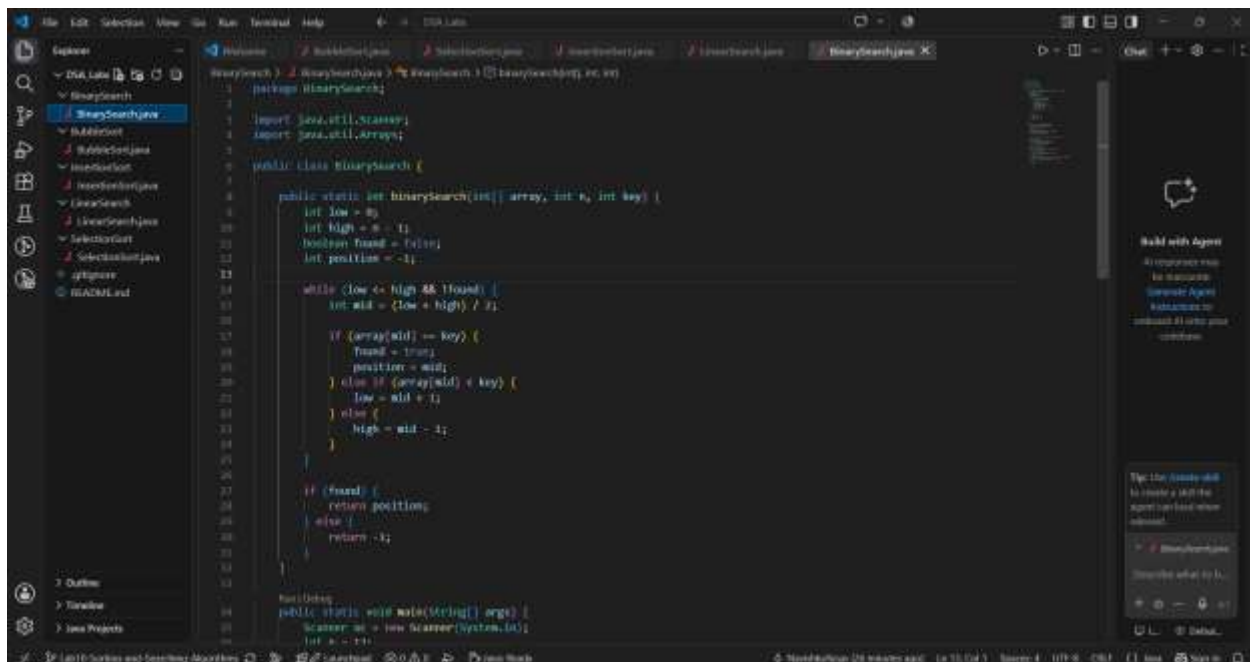
Display "Element found at position", position

Else

Display "Element not found"

7. Stop

b)c)



```
1 package BinarySearch;
2
3 import java.util.Scanner;
4 import java.util.Arrays;
5
6 public class BinarySearch {
7
8     public static int binarySearch(int[] array, int n, int key) {
9         int low = 0;
10        int high = n - 1;
11        boolean found = false;
12        int position = -1;
13
14        while (low <= high && !found) {
15            int mid = (low + high) / 2;
16
17            if (array[mid] == key) {
18                found = true;
19                position = mid;
20            } else if (array[mid] < key) {
21                low = mid + 1;
22            } else {
23                high = mid - 1;
24            }
25        }
26
27        if (found) {
28            return position;
29        } else {
30            return -1;
31        }
32    }
33
34    public static void main(String[] args) {
35        Scanner sc = new Scanner(System.in);
36        int n = 10;
37    }
38 }
```



```
1 // BinarySearch.java
2
3 // Run (Debug)
4 public static void main(String[] args) {
5     Scanner sc = new Scanner(System.in);
6     int n = 12;
7     int[] array = new int[n];
8
9     System.out.println("Enter 12 integers:");
10    for (int i = 0; i < n; i++) {
11        array[i] = sc.nextInt();
12    }
13
14    // Sort the array first (required for binary search)
15    Arrays.sort(array);
16
17    System.out.println("Sorted Array:");
18    for (int i = 0; i < n; i++) {
19        System.out.print(array[i] + " ");
20    }
21    System.out.println();
22
23    System.out.println("Enter the search key:");
24    int key = sc.nextInt();
25
26    int result = binarySearch(array, n, key);
27
28    if (result != -1) {
29        System.out.println("Element found at position: " + result);
30    } else {
31        System.out.println("Element not found");
32    }
33
34    sc.close();
35 }
```

```
PS C:\Users\Labo> cd "C:\Users\Labo" & java -cp ".\bin\*.jar" BinarySearch
Enter 12 integers:
11
34
56
67
434
23
45
67
21
45
67
26
Sorted Array:
11 21 23 34 45 45 56 67 67 76 86 434
Enter the search key:
11
```